

Intelligent Systems,
MIPT

Natural Language Processing

L10 / 28.04.26

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) Course page](#)

[!\[\]\(d66ff64371a51729ac8c1cdaa685ba6f_img.jpg\) GitHub](#)



Content

Lecture 10: Information Retrieval (IR) & Retrieval-Augmented Generation (RAG)

- Classic IR: Tf-Idf, BM-25, Inverted Index
- Dense Retrieval: cross-encoder, ColBERT
- Hybrid Retrieval
- RAG: modules, advanced techniques

Motivation

- **Hallucinations** (sometimes didn't see, sometimes despite the fact that the data is present in the model's parameters - the neural layers simply “don't quite grasp” this information)

Motivation

- **Hallucinations** (sometimes didn't see, sometimes despite the fact that the data is present in the model's parameters - the neural layers simply “don't quite grasp” this information)
- **Calibration gap** (systematically overestimate their confidence in answers that are actually incorrect)

Motivation

- **Hallucinations** (sometimes didn't see, sometimes despite the fact that the data is present in the model's parameters - the neural layers simply “don't quite grasp” this information)
- **Calibration gap** (systematically overestimate their confidence in answers that are actually incorrect)
- **Private data** (no company's corporate documents, no patients medical records, no legal discovery in the case, and events occurring after the cutoff date)

Motivation

- **Hallucinations** (sometimes didn't see, sometimes despite the fact that the data is present in the model's parameters - the neural layers simply “don't quite grasp” this information)
- **Calibration gap** (systematically overestimate their confidence in answers that are actually incorrect)
- **Private data** (no company's corporate documents, no patients medical records, no legal discovery in the case, and events occurring after the cutoff date)
- **Idea:** If we can't remove hallucinations during training and can't regularly retrain the model on fresh data, let's give it external memory for inference

Nature of hallucinations

- **Pretraining** (goal is to predict the next token with cross-entropy, not to say “I don't know”)
- **Supervised fine-tuning** (training the model to respond to <instruction → answer>. As before, there is no penalty for a confident mistake)
- **RLHF or DPO** (teaches the model to sound confident, whether she knows the answer or not)

Information retrieval pipeline

GIVEN document collection D and user query q

GOAL an ordered list $d_1, d_2, \dots, d_n$, sorted by relevance to q

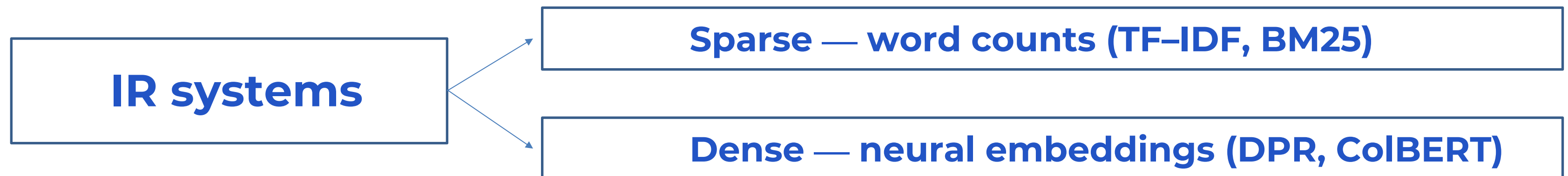
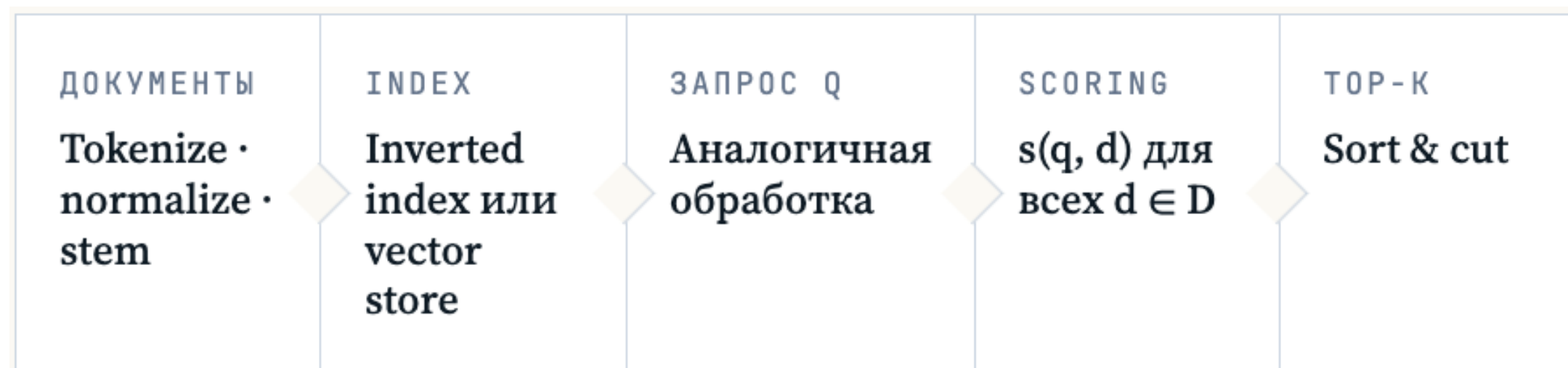
HOW train the scoring function $s(q, d)$ and maximize it on pairs with gold labels

Information retrieval pipeline

GIVEN document collection D and user query q

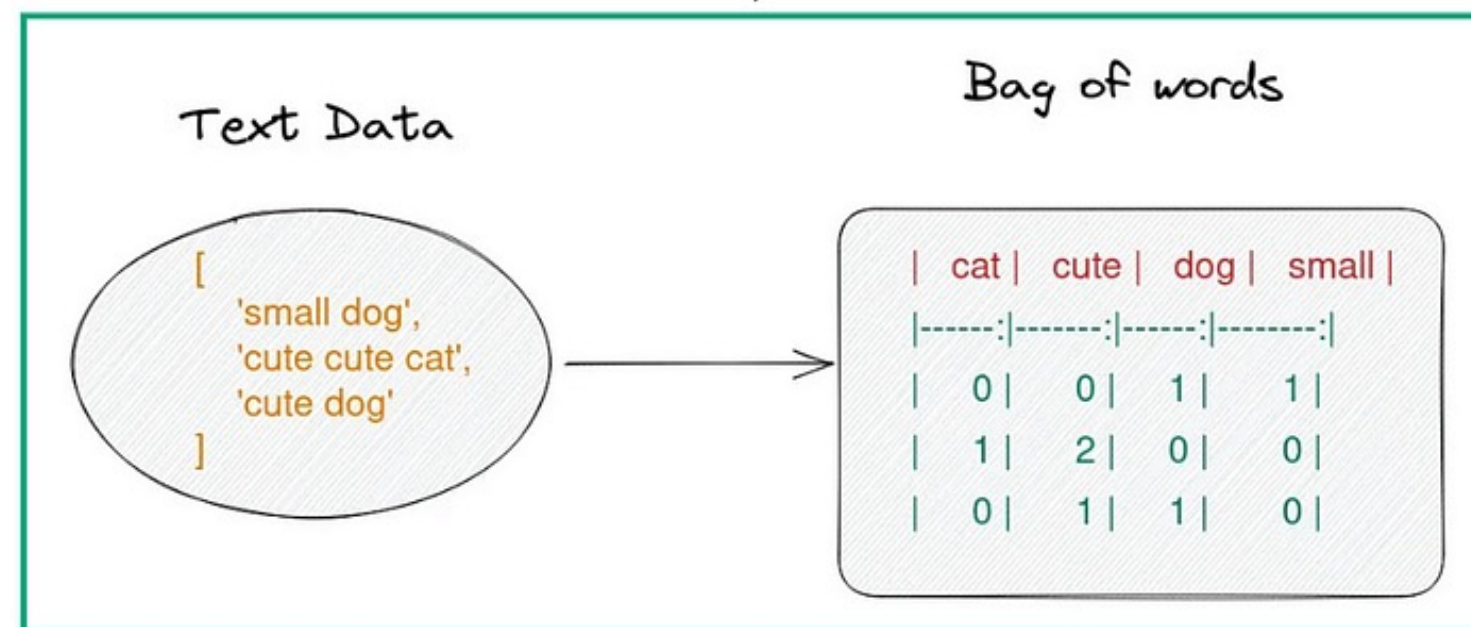
GOAL an ordered list $d_1, d_2, \dots, d_n$, sorted by relevance to q

HOW train the scoring function $s(q, d)$ and maximize it on pairs with gold labels



Bag-of-Words

- Feature matrix from classical ML
- A text, such as a paragraph or document, is presented as a set of words, and the frequency with which each word appears in the text is counted



Feature-1	Feature-2	Feature-3	Feature-4	...	...	Feature-n	
x_1^1	x_2^1	x_3^1	x_4^1	...	...	x_n^1	Sample-1
x_1^2	x_2^2	x_3^2	x_4^2	...	...	x_n^2	Sample-2
x_1^3	x_2^3	x_3^3	x_4^3	...	...	x_n^3	Sample-3
...	...	...	...	...	...	...	
x_1^m	x_2^m	x_3^m	x_4^m	...	...	x_n^m	Sample-m

Problems:

- Loss of word order → Identical vectors
"The dog bit the man", "The man bit the dog"
- Ignoring semantics → No connection between synonyms
"I drive a car", "I drive an automobile"
- The curse of dimensionality

Tf-Idf

- **Idea:** highlight words that appear frequently in this text, but rarely in other texts
- **Term Frequency - Inverse Document Frequency (TF-IDF):**

$$\text{tf}(t, d) = \frac{n_t}{\sum_k n_k}$$

$$\text{idf}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

$$\text{tf-idf}(t, d, D) = \text{tf}(t, d) \times \text{idf}(t, D)$$

where - n_t is the number of occurrences of word t in the document d , $N = |D|$

- **Score(q, d) = $\cos(v_q, v_d)$**

BM25 – industrial baseline

- **Idea:** Tf-Idf on steroids. Controls term frequency and normalizes the text to the length of the document.

- **Score(D, Q):**
$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

where k and b hyperparams.

BM25 – industrial baseline

- **Idea:** Tf-Idf on steroids. Controls term frequency and normalizes the text to the length of the document.

- **Score(D, Q):**
$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

where k and b hyperparams.

- **Why is he still alive?** Sparse methods are ideal for capturing exact matches like names, abbreviations, product codes, case identifiers, and rare terms

Ranking metrics

- > **Recall@k** — for the retrieval stage: whether the desired document made it into the top-k at all
- > **NDCG@k** — for the reranking stage: how well we ranked the results
- > **Precision@k** — for the final prompt: how clean the context is that goes into the LLM

METRIC	WHAT MEASURE
Precision@k	Proportion of relevant items among the top-k
Recall@k	Percentage of found out of all relevant results
MAP	Average AP per request
MRR	1 / rank of the first relevant result
NDCG@k	Takes into account the position with a log-discount

Dense Retrieval

QUERY

“Best cheap laptops”

DOCUMENT

“Top affordable notebooks”

BM25 cannot find a connection between “cheap” and “affordable”—there are no common tokens. Sparse methods are fundamentally at a disadvantage here.

Dense Retrieval

QUERY

“Best cheap laptops”

DOCUMENT

“Top affordable notebooks”

BM25 cannot find a connection between “cheap” and “affordable”—there are no common tokens. Sparse methods are fundamentally at a disadvantage here.

- **Idea:** encode queries and documents into embeddings using a language model. Semantically similar texts receive similar vectors unless they overlap superficially

Dense Retrieval: architectures

Cross-Encoder

Encode together, produce score directly

[CLS]query tokens[SEP]document tokens[SEP]

Transformer

(full self-attention across ALL tokens)

Q tokens

D tokens

← cross-attention →

[CLS] output

Linear → Sigmoid

0.94

relevance score

Dense Retrieval: architectures

Cross-Encoder

Encode together, produce score directly

[CLS]query tokens[SEP]document tokens[SEP]

Transformer

(full self-attention across ALL tokens)

Q tokens

D tokens

[CLS] output

Linear → Sigmoid

0.94

relevance score

Bi-Encoder

Encode separately, compare vectors

Query

Document

Transformer

(encoder only)

Transformer

(encoder only)

Mean Pool

Mean Pool

1 × 384 vector

1 × 384 vector

$\cos(q, d)$

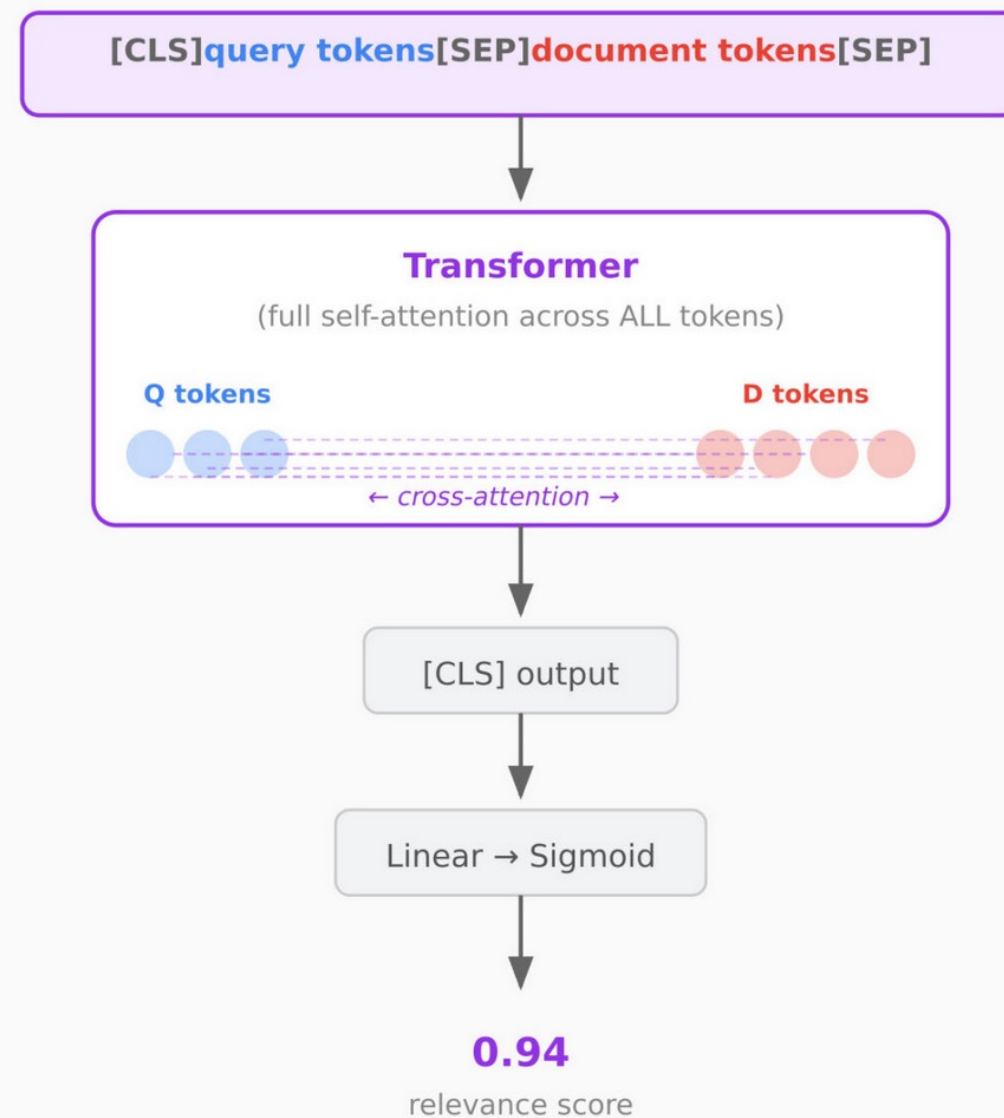
0.82

similarity score

Dense Retrieval: architectures

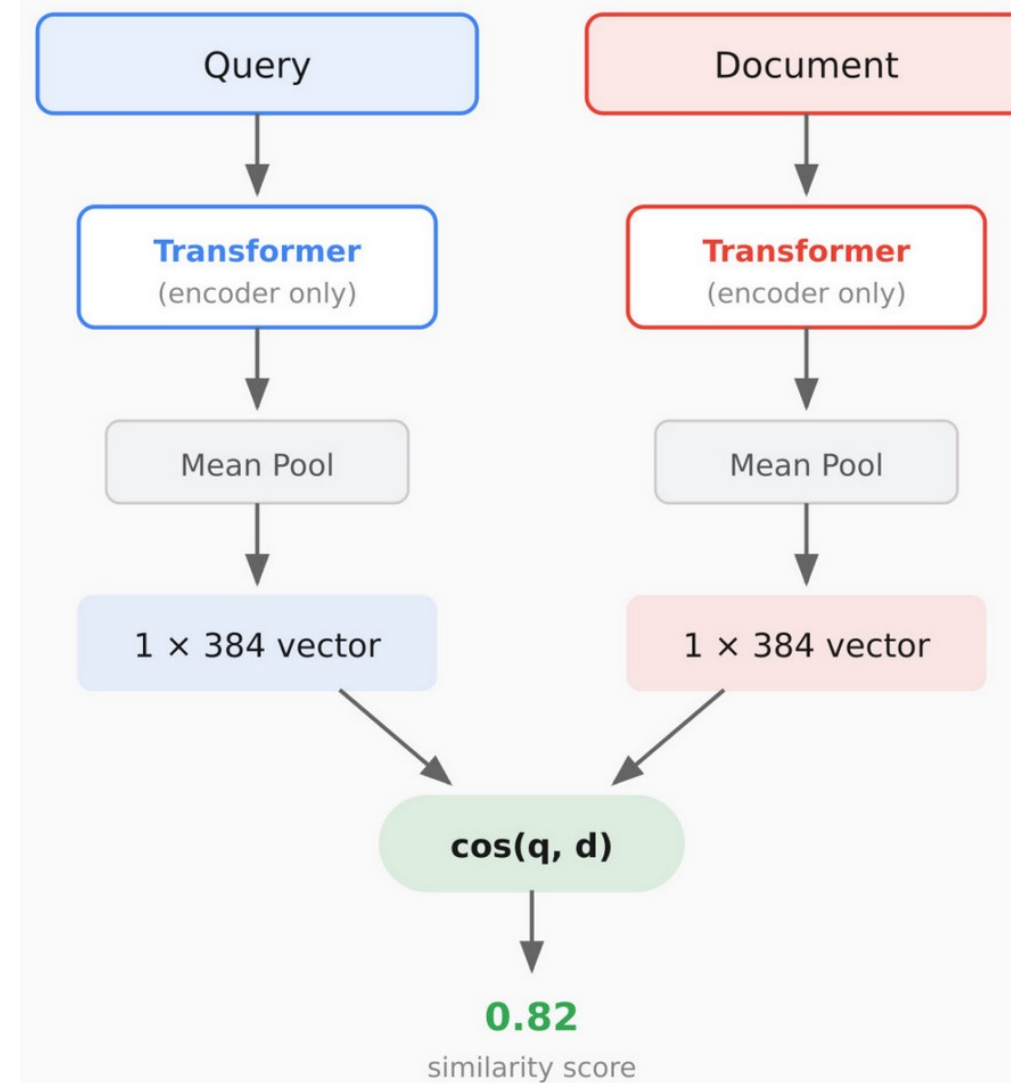
Cross-Encoder

Encode together, produce score directly

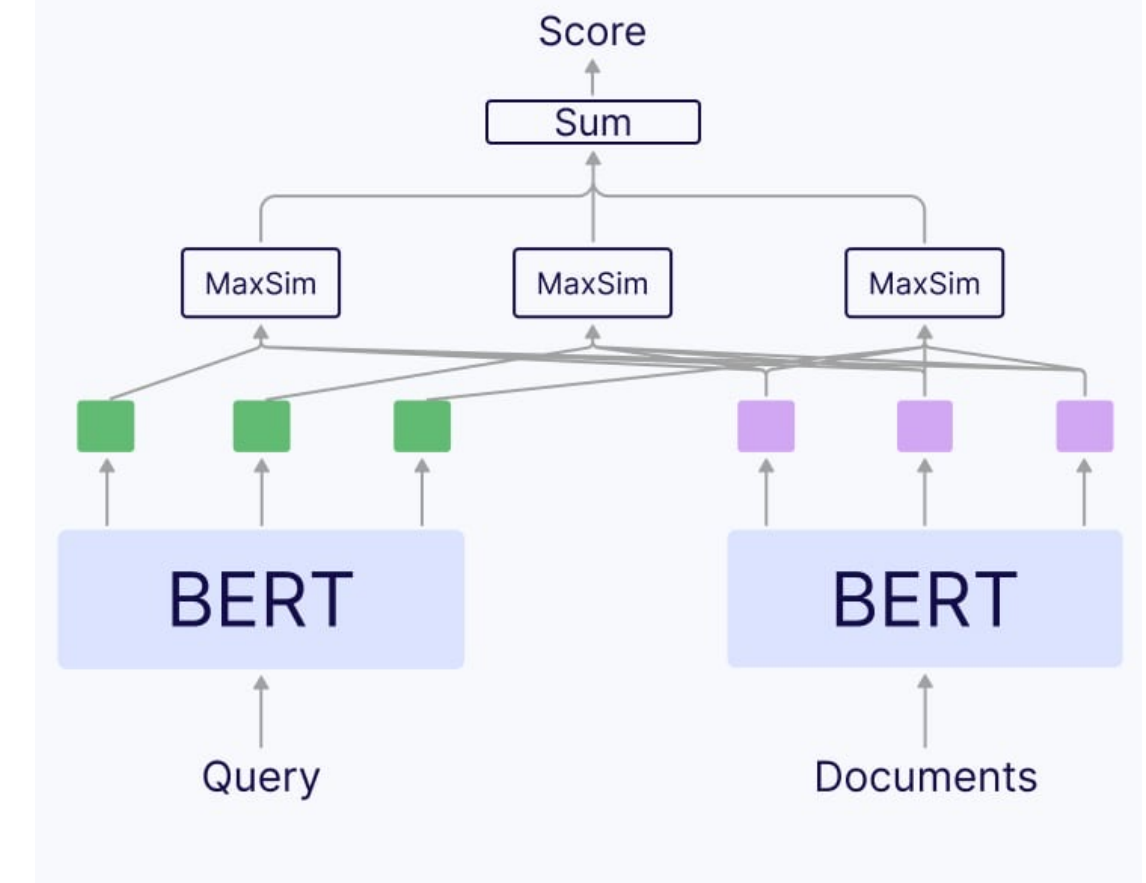


Bi-Encoder

Encode separately, compare vectors



ColBERT



Dense Retrieval: architectures

> Pros: Maximum accuracy

> Cons: $O(N)$ forward passes for query

> Pros: Very quickly (ANN search)

> Cons: No cross-attention between q & d

> Pros: Accuracy vs. Speed Trade-off

> Cons: High index (N tokens per doc)

Cross-Encoder

Encode together, produce score directly

[CLS]query tokens[SEP]document tokens[SEP]

Transformer

(full self-attention across ALL tokens)



[CLS] output

Linear → Sigmoid

0.94

relevance score

Bi-Encoder

Encode separately, compare vectors

Query

Document

Transformer
(encoder only)

Transformer
(encoder only)

Mean Pool

Mean Pool

1 × 384 vector

1 × 384 vector

$\cos(q, d)$

0.82

similarity score

CoBERT

Score

Sum

BERT

Query

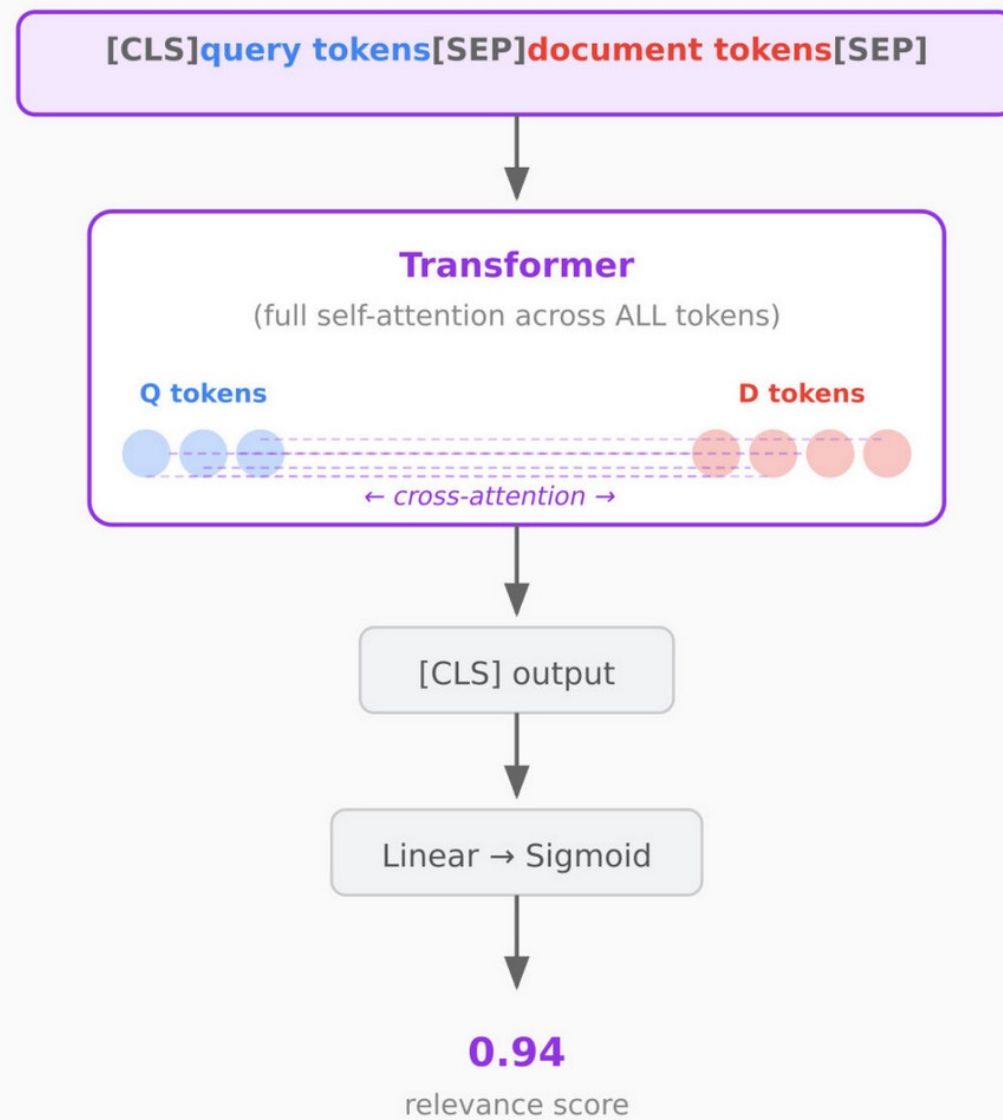
BERT

Documents

Dense Retrieval: architectures

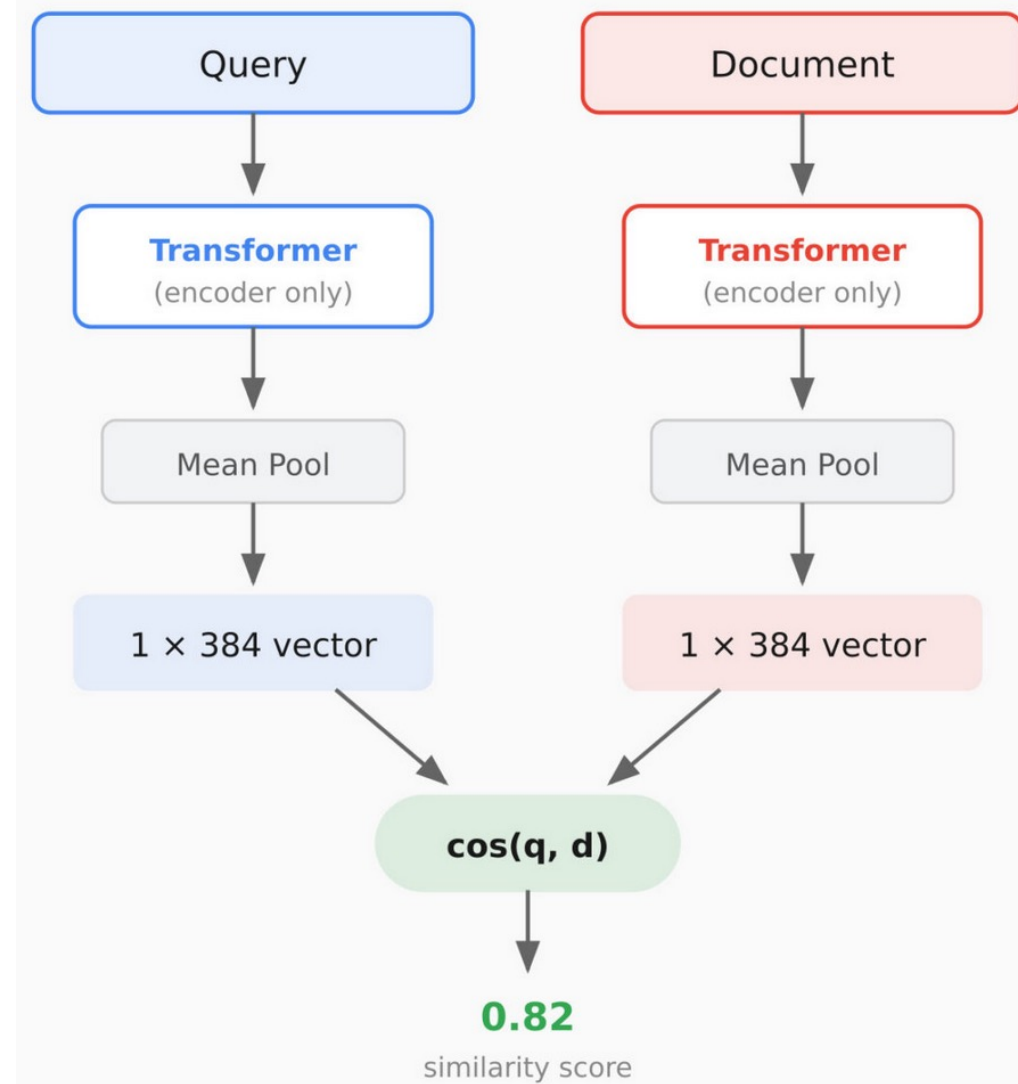
Cross-Encoder

Encode together, produce score directly

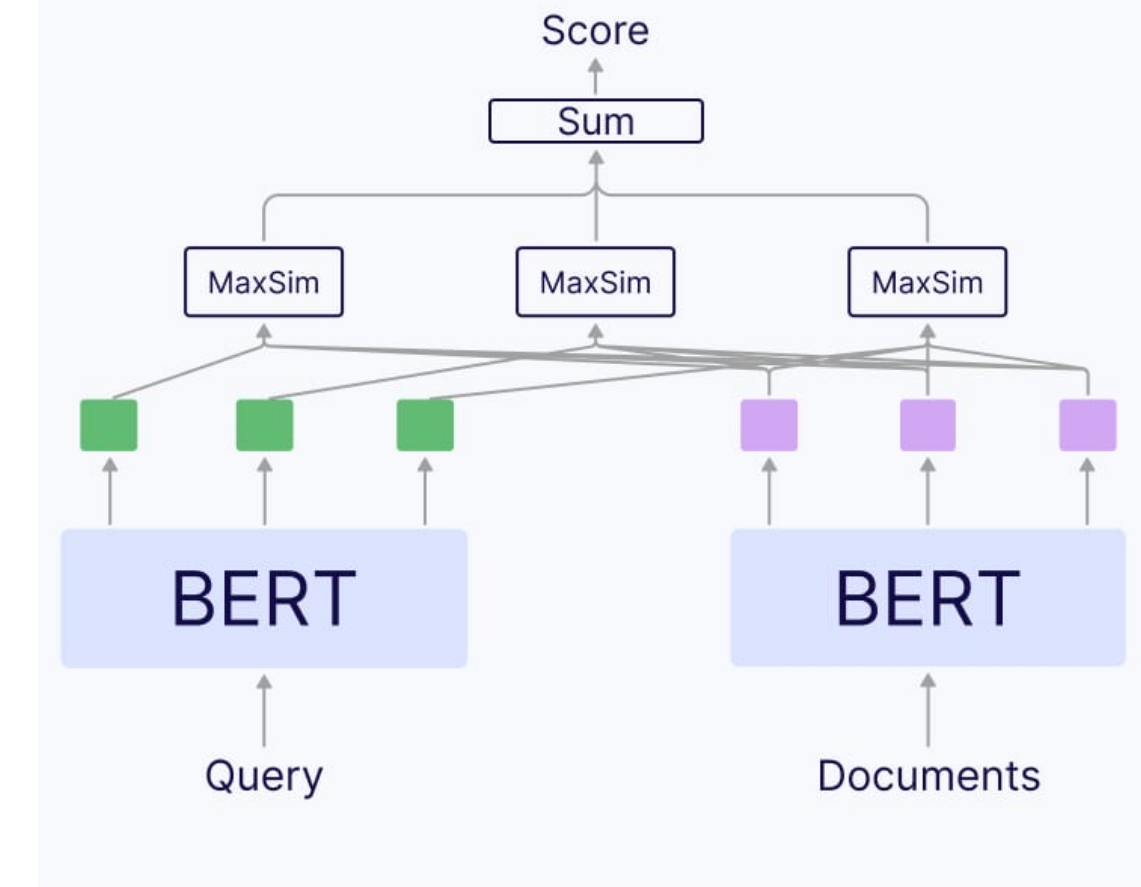


Bi-Encoder

Encode separately, compare vectors



CoBERT



- **In practice:** all three are combined, a bi-encoder for **retrieving** results from millions of items, a cross-encoder for **re-ranking** the top 100

How to train retriever?

- We could use triplet loss with margin:

$$L = \sum \max(0, \delta - \text{sim}(q, d^+) + \text{sim}(q, d^-))$$

Push the relevant d^+ closer to q , and the irrelevant d^- beyond the margin δ .

- Negative sources:

- In-batch negatives --- cheap, but not very effective. Other positives in the batch behave like negatives
- BM25 hard negatives --- the standard. We take the top results from BM25 and exclude the positives
- Mined hard negatives --- SOTA. We iteratively search for documents that are falsely similar using our own retriever

- **Practical tip:** good negatives are more important than the choice of architecture or encoder

How to search through a 1kk vectors? ---

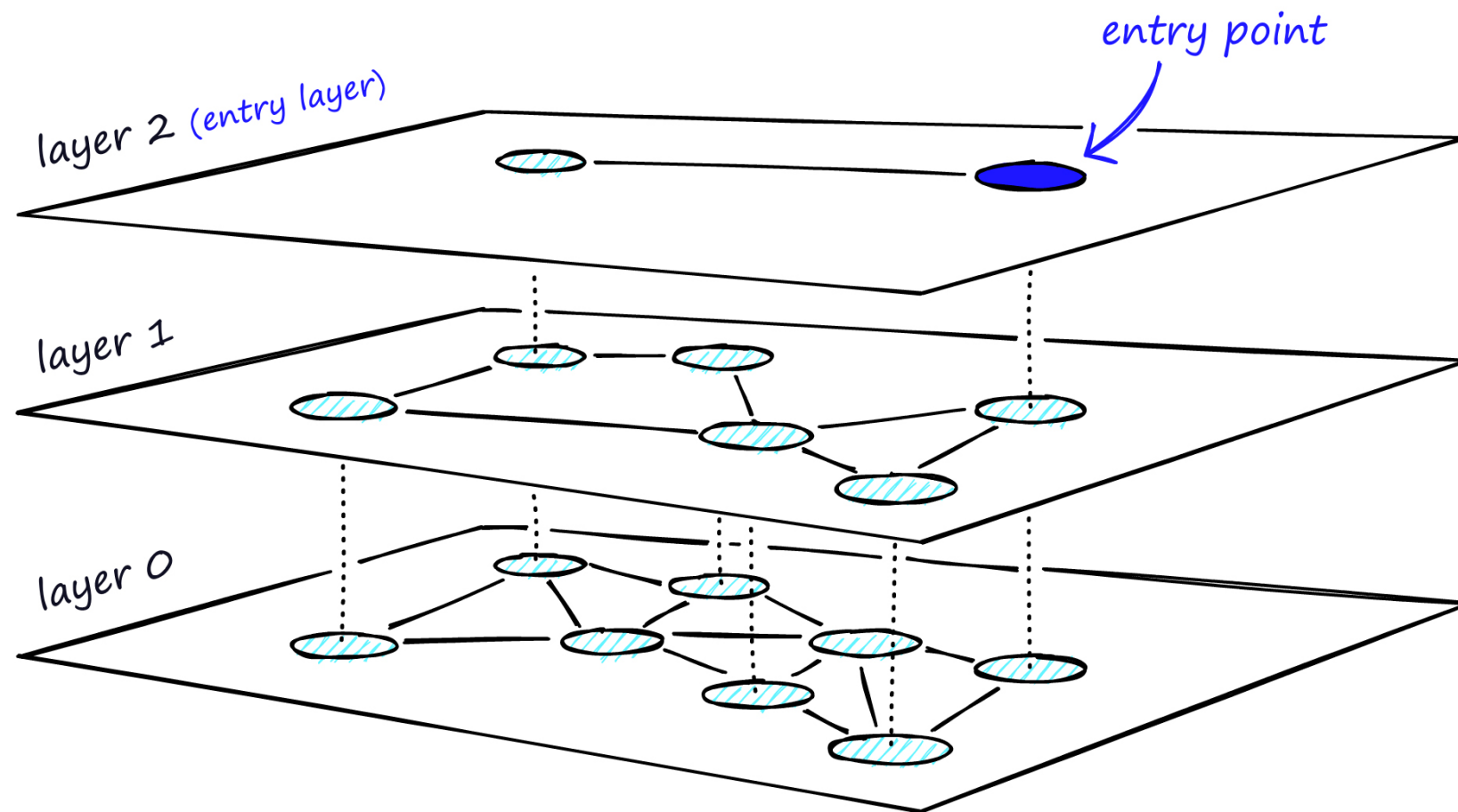
- **Problem:** after dense encoding, we have millions of 768 dimensional vectors. Exact k-NN using brute force is unacceptably slow...

How to search through a 1kk vectors?

- **Problem:** after dense encoding, we have millions of 768 dimensional vectors. Exact k-NN using brute force is unacceptably slow...

TWO MAIN FAMILIES OF ALGORITHMS:

HNSW



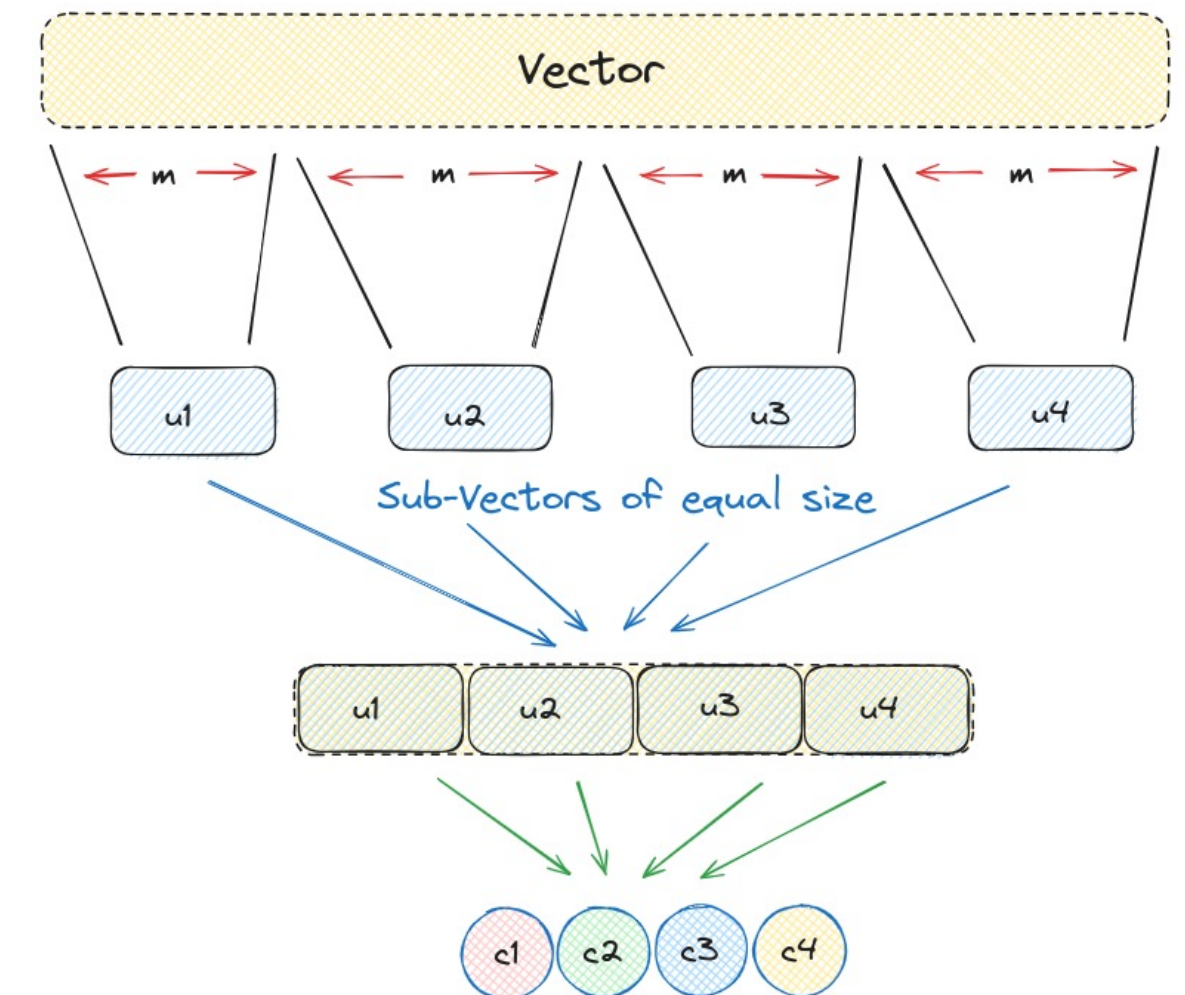
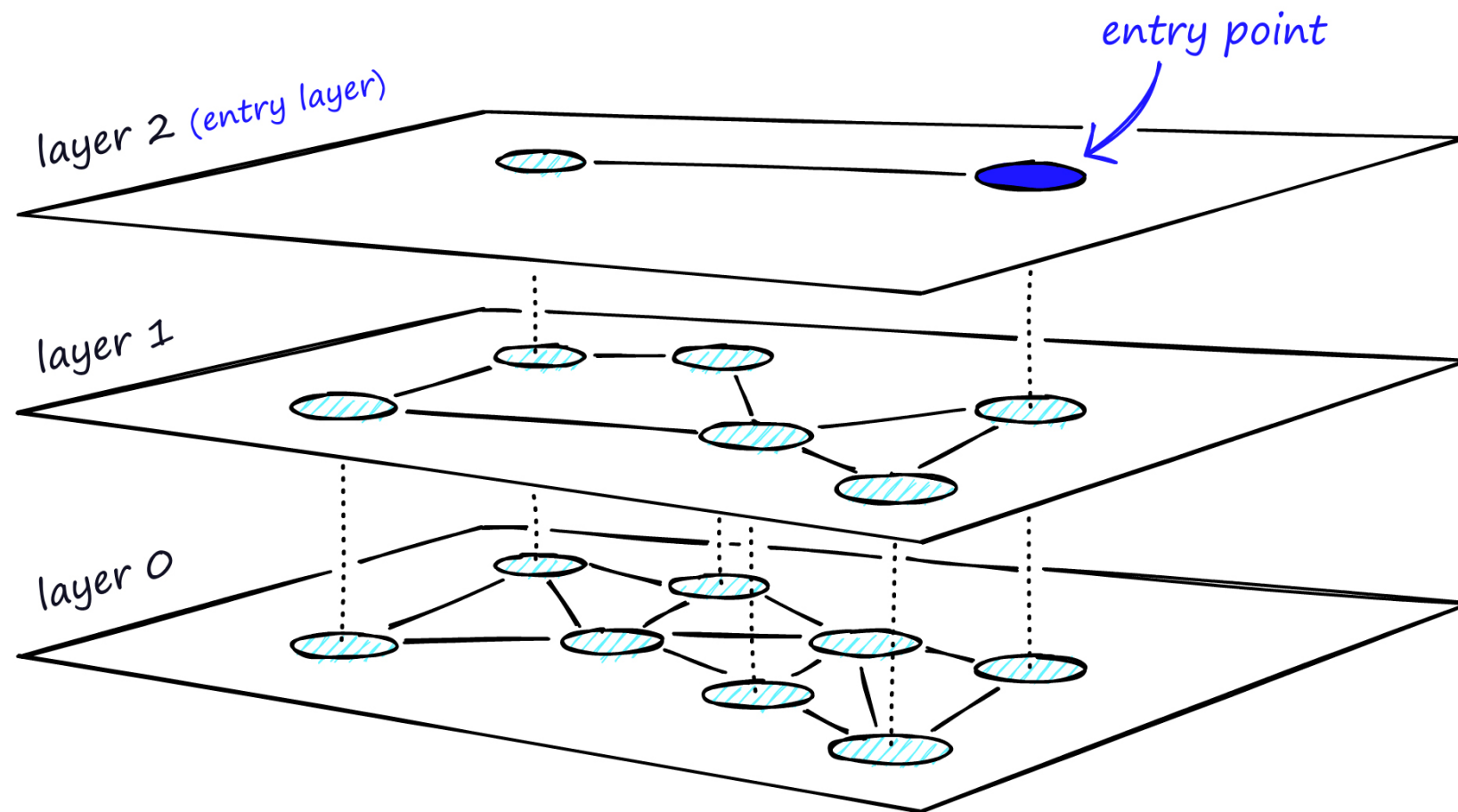
How to search through a 1kk vectors?

- **Problem:** after dense encoding, we have millions of 768 dimensional vectors. Exact k-NN using brute force is unacceptably slow...

TWO MAIN FAMILIES OF ALGORITHMS:

HNSW

IVF-PQ



Hybrid Retrieval

- Sparse and dense methods catch different errors. Errors are weakly correlated → the ensemble actually improves performance rather than blurring the signal

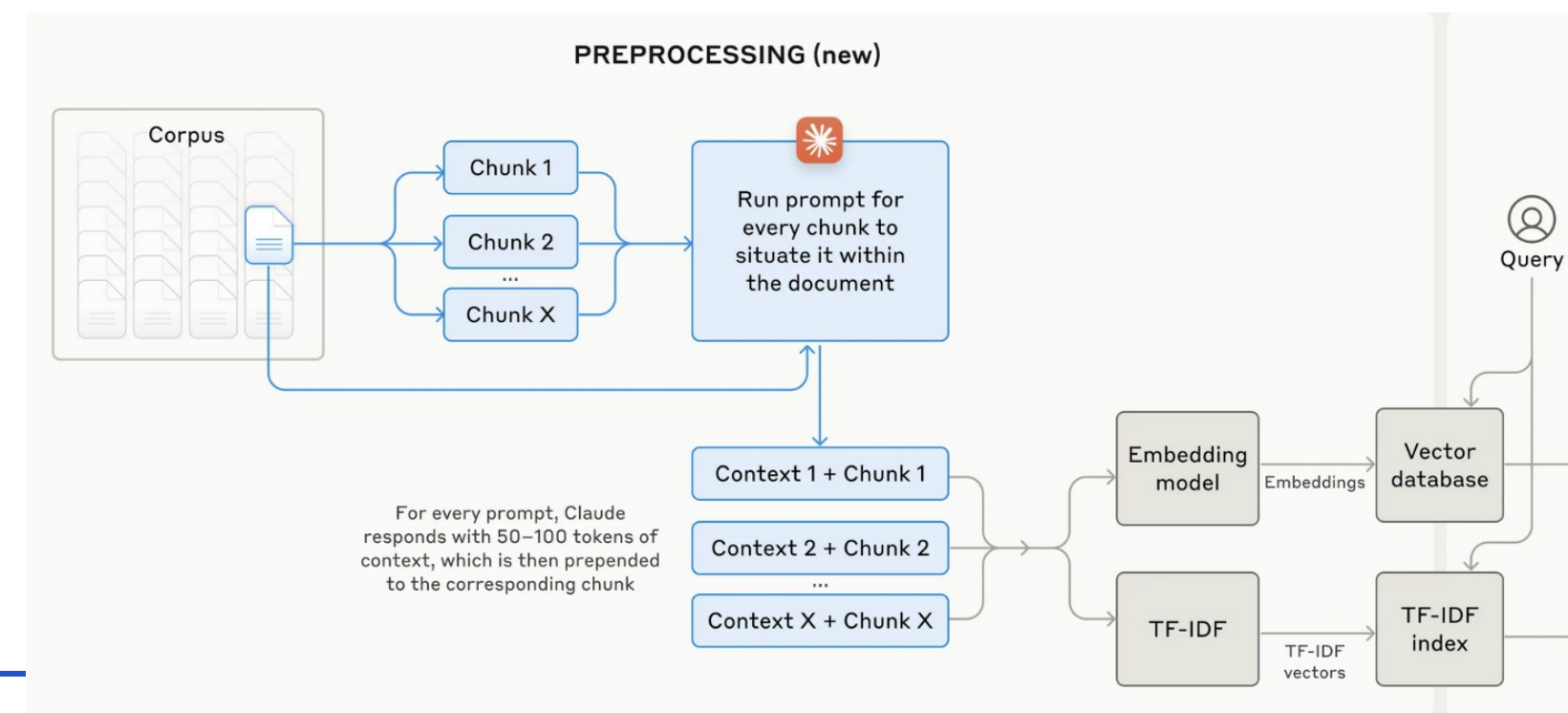
	What it captures	Where it fails 
Sparse BM25	Exact matches: names, abbreviations, SKUs/part numbers, rare terms	Paraphrases, synonyms, cross-lingual matches
Dense DPR	Semantics, paraphrases, cross-lingual correspondences	Exact names/codes, out-of-domain terms

Hybrid Retrieval

- Sparse and dense methods catch different errors. Errors are weakly correlated → the ensemble actually improves performance rather than blurring the signal

	What it captures	Where it fails	⬇
Sparse BM25	Exact matches: names, abbreviations, SKUs/part numbers, rare terms	Paraphrases, synonyms, cross-lingual matches	
Dense DPR	Semantics, paraphrases, cross-lingual correspondences	Exact names/codes, out-of-domain terms	

- Real-world example: Anthropic Contextual Retrieval



Reciprocal Rank Fusion

- Problem: **BM25** and **cosine scores** are not comparable on the same scale. Direct summation will break down. RRF uses only ranks, so scale does not matter

Reciprocal Rank Fusion

- Problem: **BM25** and **cosine scores** are not comparable on the same scale. Direct summation will break down. RRF uses only ranks, so scale does not matter
- Idea: We use only ranks, not raw scores

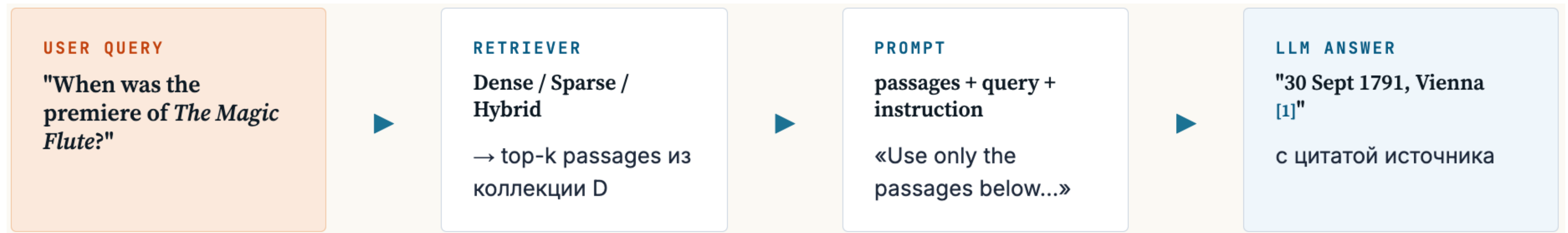
$$\text{RRF}(d) = \sum_i \frac{1}{k + \text{rank}_i(d)}$$

where k - hyperparameter, usually = 60, i - ranking method

- Real-world example: Elasticsearch, OpenSearch, Vespa, Weaviate

RAG intro

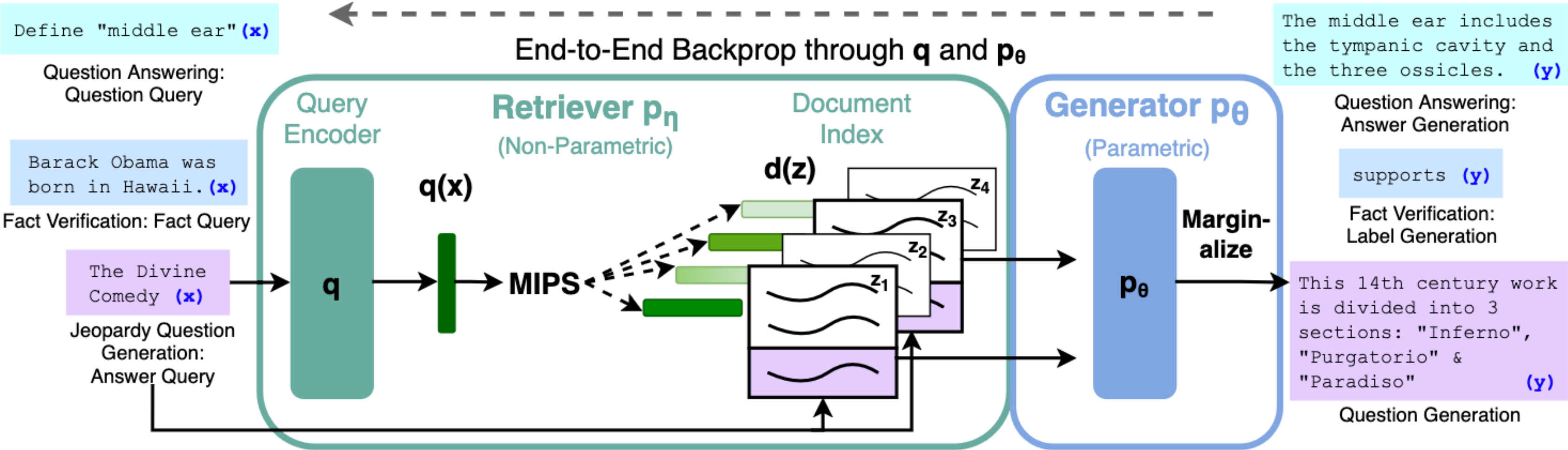
- Base RAG = retriever + generator



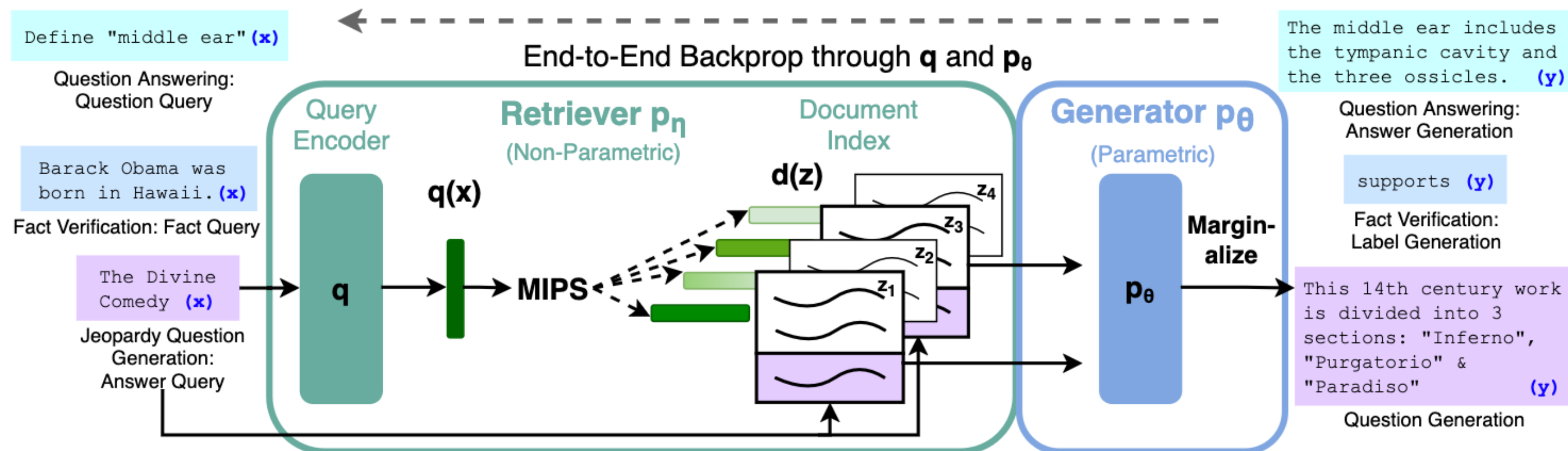
- $p(x_1, \dots, x_n) = \prod_i p(x_i | R(q); q; x_{<i})$, where $R(q) = d_1 \dots d_k$ — top-k passages from collection D

Each response token is conditionally dependent on: the retrieved context $R(q)$, the query q , and the previously generated tokens $x_{<i}$. This is an autoregressive generation process, like a standard LLM, but with additional conditioning on the retrieved passages.

Joint RAG



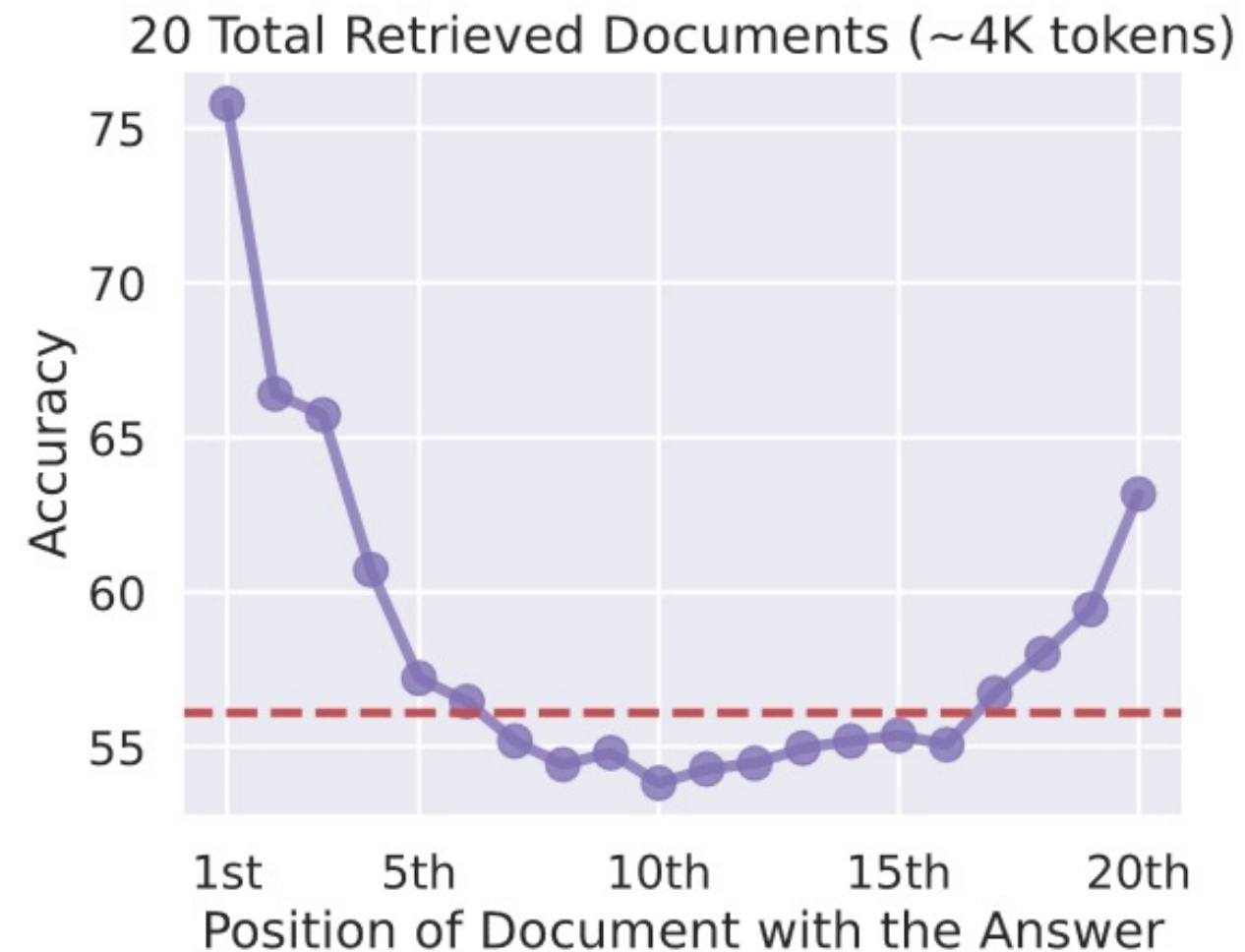
Joint RAG



- **Problem:** end-to-end backpropagation via a retriever and a generator
- **What will win?** retriever and the generator are independent components, each one improves on its own

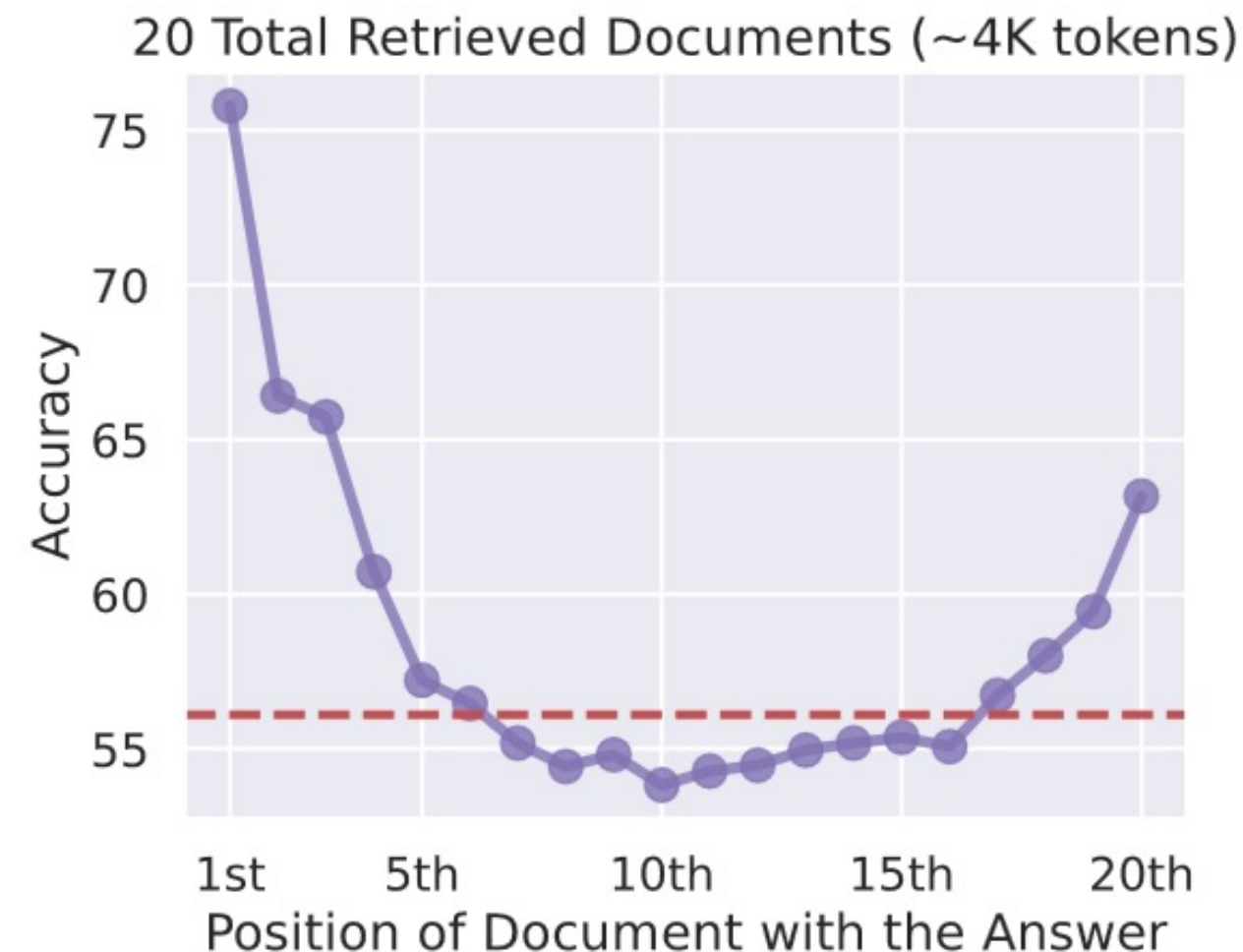
Lost in the Middle

- **TLDR:** The position where you put the document in LLM matters for the quality of responses



Lost in the Middle

- **TLDR:** The position where you put the document in LLM matters for the quality of responses



- **Relevance gap:** Retriever is optimized for IR metrics (precision and recall). Generator requires documents containing actual evidence to generate a response. Their objectives are not the same.

Latency budget

- Any “advanced” technique that involves an additional LLM call doubles latency. This is an engineering trade-off that’s worth keeping in mind when designing a pipeline



RAG query

= 1-3 s

Advanced RAG: pre-retrieval

- **Query Transformations**

- > **HyDE (Hypothetical Document Embeddings)** --- ask LLM to generate a hypothetical response document, as if it were responding to the query
- > **Query Expansion** --- instead of a single prompt, we ask the LLM to generate **N** paraphrases
- > **Query Decomposition** --- break down a complex query into subqueries

Advanced RAG: pre-retrieval

- **Query Transformations**

- > **HyDE (Hypothetical Document Embeddings)** --- ask LLM to generate a hypothetical response document, as if it were responding to the query
- > **Query Expansion** --- instead of a single prompt, we ask the LLM to generate **N** paraphrases
- > **Query Decomposition** --- break down a complex query into subqueries

- **Chunking & Indexing**

- > **Hierarchical indexing** --- Document → Sections → Chunks
- > **Semantic Chunking** --- Instead of a fixed size of 500 tokens, we split the text along semantic boundaries
- > **Multi-representation indexing** --- chunk is stored in the index in multiple views

Advanced RAG: post-retrieval

- Two-stage reranking

➤ The bi-encoder, or BM25 — an inexpensive retriever — returns a large number of candidates, typically 100.

The cross-encoder — an expensive reranker — reorders them into a top 10 list. This top 10 list is then fed into the LLM as a prompt.



Advanced RAG: post-retrieval

- **Two-stage reranking**

> The bi-encoder, or BM25 — an inexpensive retriever — returns a large number of candidates, typically 100. The cross-encoder — an expensive reranker — reorders them into a top 10 list. This top 10 list is then fed into the LLM as a prompt.



- **Context compression**

- > **LongLLMLingua** --- LLM-based filtering at the sentence level
- > **xRAG** --- compress each retrieved document into a single token using a specially trained adapter
- > **Contextual filtering** --- pass each sentence through a relevance classifier, conditioned on the query

Frontier RAG

ASAI ET AL. • 2023

Self-RAG

LLM is trained to generate reflection tokens:

whether retrieval is needed, whether the document is relevant, and whether the statement is supported.

A single LLM manages the entire pipeline — it decides on its own when and what to search for.

MICROSOFT • 2024

GraphRAG

Instead of a flat vector index, construct a knowledge graph using LLM-based extraction of entities and relationships. When a query is received, we traverse the graph and retrieve subgraphs and community summaries. Multi-hop reasoning is built-in.

2024 +

Agentic RAG

Retrieval is one of the tools available to an autonomous agent. The agent plans when and how to search, performs several rounds of retrieval, and reformulates the query based on the interim results.

Frontier RAG

ASAI ET AL. • 2023

Self-RAG

LLM is trained to generate reflection tokens: whether retrieval is needed, whether the document is relevant, and whether the statement is supported. A single LLM manages the entire pipeline — it decides on its own when and what to search for.

MICROSOFT • 2024

GraphRAG

Instead of a flat vector index, construct a knowledge graph using LLM-based extraction of entities and relationships. When a query is received, we traverse the graph and retrieve subgraphs and community summaries. Multi-hop reasoning is built-in.

2024 +

Agentic RAG

Retrieval is one of the tools available to an autonomous agent. The agent plans when and how to search, performs several rounds of retrieval, and reformulates the query based on the interim results.

- “RAG is dead, long context wins” — that didn’t happen. **In practice:** long context is good for anaphora and cross-document reasoning; RAG is good for latency, cost, and precision on large corpora. The “Lost in the Middle” problem hasn’t gone away, even with Gemini and its 2M context. Today, these are complementary, not competing, techniques.

Recap

- RAG isn't magic — it's IR combined with LLM conditioning. Each component degrades independently
- BM25 is the best of the best. A sparse-dense hybrid is almost always better than any single approach
- The main unresolved issue is the relevance gap between the retriever and the generator. All advanced techniques address this gap.
- 80% of the quality of production-RAG comes from effective chunking and reranking, not from the choice of embedder

Next:

- Agents